DigiPen
INSTITUTE OF TECHNOLOGY
SINGAPORE

Local AI

Running Your Own LLM

From previous AI apps to local AI

Same application patterns, different place where the model runs.

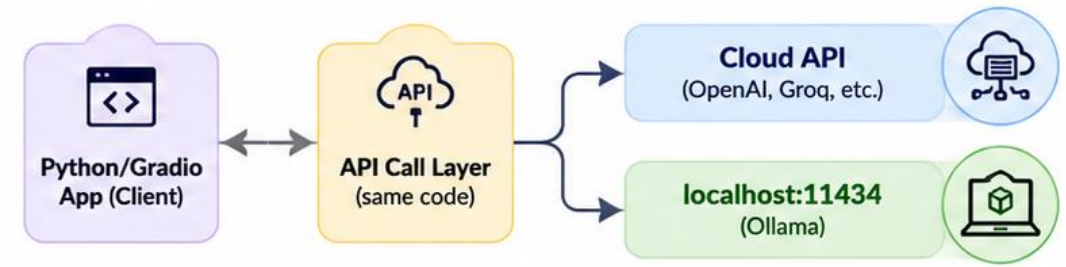
Before: cloud-AI applications

Your Python/Gradio/RAG app sent prompts to a hosted model API.
The cloud model handled compute, scaling, and model storage.
You focused on prompts, retrieval, memory, and UI.

Now: local inference

The same app can call a model server on your machine: **localhost:11434** *

This changes cost, privacy, latency, model quality, and hardware responsibility.



* **11434** is the default port used by Ollama

The Local AI Landscape in 2025

Just few years ago, running an LLM required a data center. Today, a simple laptop can run a 4-billion parameter model that beats 2021's GPT-3 in many tasks.

2020: Mostly Cloud

- GPT-3 required massive infrastructure
- Mostly API-based access
- Limited open-weight models
- Data often sent to external servers
- API costs per request

2024: Local AI Becomes Practical

- Modern small open models available
- Easy setup with Ollama
- Can run on consumer laptops
- Private/offline inference possible
- No per-request API fees

Tools Available Now

- Ollama (easiest)
- LM Studio (GUI)
- llama.cpp (fastest)
- LocalAI (server)
- Jan (desktop app)
- ...


VS

Local vs Cloud AI

Choose the right approach for each use case, they're complementary, not competing

LOCAL AI

Run on your own machine / server

- ✓ Full data privacy — data never leaves
- ✓ Zero per-token cost after setup
- ✓ Works completely offline
- ✓ No rate limits or quotas
- ✗ Slower on consumer hardware
- ✗ Lower quality than frontier models

VS

CLOUD AI

API calls to remote model servers

- ✓ State-of-the-art model quality
- ✓ No hardware requirements
- ✓ Instant scale, auto-updates
- ✓ Multimodal & advanced features
- ✗ Data leaves your environment
- ✗ Costs grow with usage

When to Use Local Models

Key scenarios where local AI is the right choice

Sensitive Data

Healthcare, legal, finance — regulated data that cannot leave your premises

Cost Control

High-volume processing — millions of requests/day makes API costs prohibitive

Low Network Delay

No API round-trip time.
Performance depends on local hardware.

Offline Access

Air-gapped environments, remote locations, or no-internet deployments

Dev & Testing

Rapid iteration without API costs — test prompts and fine-tuned models freely

Custom Models

Fine-tuned or domain-specific models that are not available via public APIs

Local AI Trade-offs: Cost, Performance & Privacy

Most decisions are a negotiation between cost, performance, and privacy.

💰 COST

- Initial hardware investment
- One-time model download
- Electricity ~$0.01/hr (GPU)
- No per-token charges ever
- Cloud: ~$0.002/1K tokens
- Break-even: ~2M tokens

⚡ PERFORMANCE

- 7B model: 20-40 tok/s (GPU)
- 7B model: 3-8 tok/s (CPU)
- Quantized models faster/smaller
- Quality ≈ 60-80% of GPT-4
- Context: 4K-128K tokens
- First token latency: 0.1-2s

🔒 PRIVACY

- 100% data stays on-device
- No training on your data
- HIPAA/GDPR compliant path
- No API key exposure risk
- Audit every inference
- Air-gap deployable

Use cases for local AI

Most of the time Local AI is strongest when control and privacy matter more than absolute model strength.

Sensitive data assistant

Internal docs, Private documents, HR policies, meeting notes, internal workflows.

[Internal company assistant]

Developer sandbox

Analyze local repositories, internal codebases, or Prototype prompts/apps without spending API budget

[Private coding assistant]

Edge/offline AI

Factory floor, field laptop, lab demo with poor/unstable internet

Teaching lab

Students can see the whole system: model server + app + benchmark

High-volume automation

Document processing or internal automation where API costs become expensive at scale

Hybrid fallback

Use local for simple tasks, cloud only when quality is insufficient

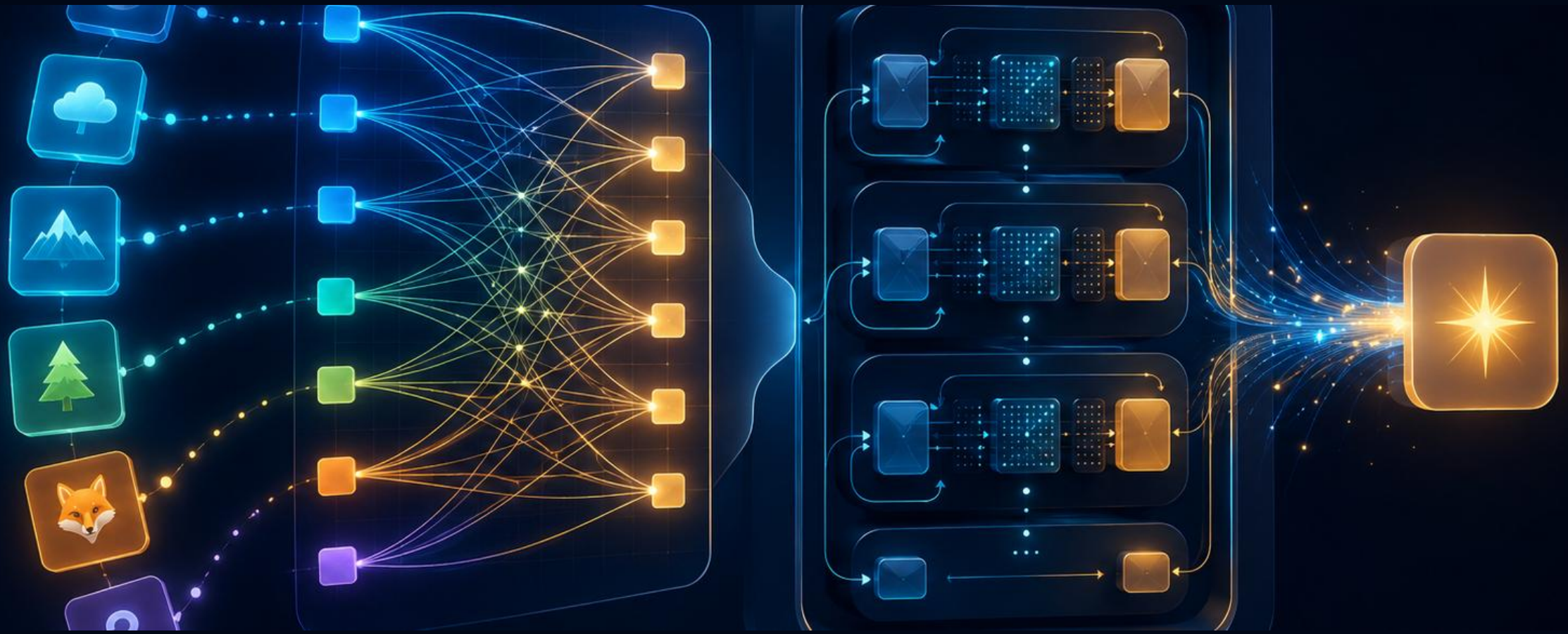
What actually runs locally?

Think of local AI as a mini model-serving stack.

Your App	Gradio / Python / Node.js / RAG or any chatbot application
API Layer	Local HTTP API requests (e.g., localhost:11434 in Ollama)
Model Runner	Loads the model, performs inference, tokenizes input, and streams generated tokens through a local API
Model Weights	Qwen / DeepSeek / Llama / Gemma files on disk stored and loaded into RAM/VRAM during inference
Hardware	CPU, RAM, GPU/VRAM, storage, operating system

Key concepts refresh: tokens, embeddings, and context windows

Important distinction
Embedding models create vectors for retrieval. Chat/generative models produce text. Some model families provide both, but they are used differently.



Input tokens → attention links → transformer layers → next-token probabilities

Types of Generative AI Models

LLMs are one part of the wider generative AI family.

Transformer LLMs

Text Generation

- GPT, Llama, Qwen, DeepSeek
- Autoregressive — predict next token
- Best for chat, code, reasoning

```
Example: ollama run qwen3:4b
```

Diffusion Models

Image / Audio

- Stable Diffusion, DALL-E, Sora
- Add noise, then denoise iteratively
- Best for image/video/audio gen
- Not covered in Ollama (yet)

VAEs

Latent Encoding

- Variational Autoencoders
- Encode data to latent space
- Used in image compression, anomaly
- Component inside diffusion models

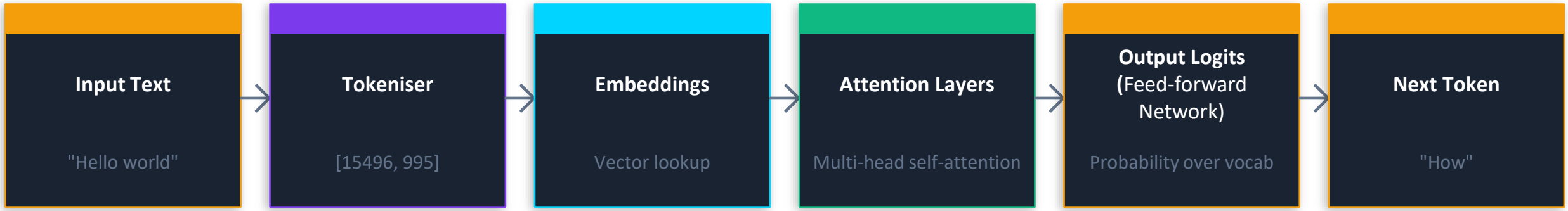
GANs

Adversarial Gen

- Generator vs Discriminator
- Generator tries to fool discriminator
- High-quality images, style transfer
- Mostly superseded by diffusion

How Transformers Work (Conceptual)

The architecture behind GPT, LLaMA, Qwen, DeepSeek...



Tokenisation

Text → sub-word units with an ID number.
'unhappy' → ['un','happy'] = [2090, 6380]
Vocab size: 32K–128K. Affects cost & context.

Embeddings

Token IDs → dense vectors (4096+ numbers).
Captures semantic meaning mathematically.
'king' - 'man' + 'woman' ≈ 'queen'

Self-Attention

Each token scores relevance against all others.
'bank' near 'river' vs near 'money' = different meaning.
This is what makes LLMs understand context.

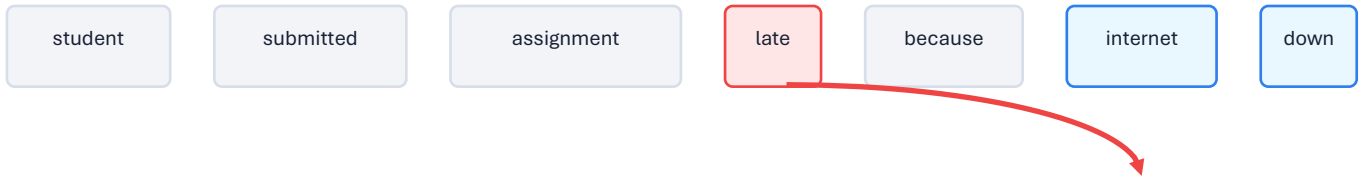
Context Window

Max tokens processed in one call.
Qwen3:4b = 32K tokens (~24K words)
Larger context = more RAM needed per call.

Attention intuition: “which words should I look at?”

Attention is a relevance mechanism inside the model.

The student submitted the assignment late because the internet was down.



“late” attends strongly to the explanation: “internet was down”

In LLMs, attention is repeated across many layers and heads; each head can learn a different relation: topic, grammar, references, code structure, or reasoning clue.

Ollama

One of the easiest way to run open-source LLMs locally

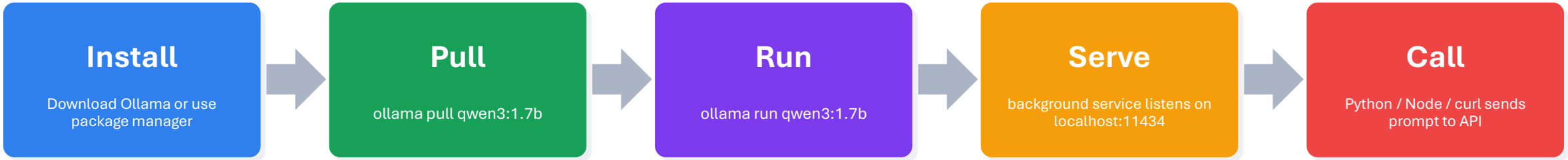
What is Ollama?

- Open-source tool to download & run LLMs with one command
- Manages model storage, GGUF format, and GPU/CPU routing
- Exposes a REST API on localhost:11434 — drop-in for OpenAI SDK
- Supports macOS (Metal), Windows (CUDA/CPU), Linux (CUDA/ROCm)
- Models: Qwen3, DeepSeek-R1, LLaMA, Mistral, Phi, Gemma and more

```
$ terminal

# Install (macOS/Linux)
curl -fsSL https://ollama.com/install.sh | sh

# Pull a model
ollama pull qwen3:0.6b
ollama pull deepseek-r1:7b

# Run interactively
ollama run qwen3:0.6b

# Check running models
ollama list
```

Architecture: Ollama runs as a daemon on port 11434. The CLI communicates with the daemon via REST. This means the same API works from your terminal, Python, Node.js, or any HTTP client.

Ollama




```
ollama run qwen3:0.6b

>>> Explain RAG to a beginner in 4 bullet points.

>>> Now answer using an analogy with a library.

/bye
```

```
ollama run qwen3:4b

Repeat the same prompt.
Compare:
- response time
- correctness
- clarity
- hallucination risk
```

Ollama API: replacing the cloud API call

Verify Ollama is running and both models are installed before anything else

```python
import requests

response = requests.post(
    "http://localhost:11434/api/chat",
    json={
        "model": "qwen3:0.6b",
        "stream": False,
        "messages": [
            {"role": "system", "content": "You are a helpful tutor."},
            {"role": "user", "content": "Explain local AI simply."}
        ]
    },
    timeout=120
)

print(response.json()["message"]["content"])
```

What changed?
Cloud version: send request to provider endpoint with an API key. Local version: send request to localhost where Ollama is running.

What did not change?
Prompt structure, messages, memory, RAG snippets, and UI code can remain almost the same.

⚠️ **If you see an error:**
ollama serve (then retry)

`practice_01_check_ollama.py`

Local document assistant architecture

Your previous RAG app can become local by swapping the generator model.

Retrieval can remain the same:
load files → split chunks → store/search embeddings. Only the generation step changes from cloud model to local model.

Important warning: local generation does not mean local embeddings unless your embedding model is also local.

local document Q&A

After checking Ollama, each snippet adds one missing part of the RAG pipeline

```
SUPPORTED_EXTENSIONS = {".txt", ".md", ".py", ".csv", ".html", ".pdf",
".docx"}

def load_one_file(path: Path) -> dict:
    suffix = path.suffix.lower()

    if suffix in {".txt", ".md", ".py", ".csv", ".html"}:
        text = read_text_file(path)
    elif suffix == ".pdf":
        text = read_pdf_file(path)
    elif suffix == ".docx":
        text = read_docx_file(path)
    else:
        raise ValueError(f"Unsupported file type: {path.suffix}")

    return {
        "filename": path.name,
        "path": str(path),
        "extension": suffix,
        "text": text,
        "characters": len(text),
    }
```

What changed?
Instead of typing one prompt, we first collect knowledge from local files. The model cannot use a document unless we read it into text first.

PDF and DOCX need extra libraries: pypdf and python-docx. Plain text files can be read directly with Python.

⚠️ If nothing loads
Check that the docs folder exists and the files have supported extensions.

`practice_02_load_many_files.py`

Before embedding: why do we chunk documents?

A long document is too large and too vague, so we split it into smaller overlapping pieces

```
def chunk_text(text: str, chunk_size: int = 900, overlap: int =
150) -> list[str]:
    clean = " ".join(text.split())
    chunks = []
    start = 0

    while start < len(clean):
        end = start + chunk_size
        chunk = clean[start:end]
        if chunk.strip():
            chunks.append(chunk.strip())
        start += chunk_size - overlap

    return chunks



def embed_text(text: str) -> list[float]:
    payload = {"model": EMBED_MODEL, "input": text}
    response = requests.post(
        f"{OLLAMA_BASE_URL}/api/embed",
        json=payload,
        timeout=120
    )
    response.raise_for_status()
    embeddings = response.json().get("embeddings", [])
    return embeddings[0]

records.append({
    "id": f"{doc['filename']}::chunk_{index:04d}",
    "filename": doc["filename"],
    "path": doc["path"],
    "chunk_index": index,
    "text": chunk,
    "embedding": vector,
})
```

1 Chunk size
How much text goes into each searchable piece.

2 Overlap
Repeats the end of one chunk at the start of the next chunk so meaning is not cut off.

3 Result
The vector store saves many small chunks, not one huge document.

Each record keeps: file name, path, chunk number, original chunk text, and the embedding vector.

This is not the fastest vector database, but it is transparent.

`practice_03_chunk_embed_store.py`

Retrieve the most relevant chunks

Goal: test whether the vector store can find useful chunks before asking the LLM

```
def retrieve(query: str, store_path: str = "local_vector_store.json", top_k:
int = 4) -> list[dict]:
    records = json.loads(Path(store_path).read_text(encoding="utf-8"))
    query_vector = embed_text(query)

    scored = []
    for record in records:
        score = cosine_similarity(query_vector, record["embedding"])
        scored.append((score, record))

    scored.sort(key=lambda item: item[0], reverse=True)

    results = []
    for score, record in scored[:top_k]:
        record = dict(record)
        record["score"] = score
        results.append(record)

    return results
```

The question is embedded too. Then we compare the question vector with every stored chunk vector.

What does top_k mean?
Return only the best few chunks. More chunks give more context, but also more noise and slower answers.

✅ Top results show the expected file, chunk number, score, and preview text.

This is retrieval only — no final answer yet.

`practice_04_retrieve_test.py`

build a grounded local document Q&A CLI

Combine retrieval + a strict prompt + local chat model

```
system = (
    "You are a careful local document assistant. "
    "Answer using ONLY the provided sources. "
    "If the answer is not in the sources, say: "
    "'I could not find this in the provided documents.' "
    "Cite sources using [Source 1], [Source 2], etc."
)

user = f"""Question:
{question}

Retrieved document sources:
{context}

Answer:"""

def answer_question(question: str, top_k: int = 4):
    chunks = retrieve(question, top_k=top_k)
    messages = build_grounded_prompt(question, chunks)
    answer = ask_ollama(messages)
    return answer, chunks
```

Now the LLM receives the question plus the retrieved chunks. It is instructed to answer only from the sources.

High similarity does not guarantee truth. It only means the chunk looks related. The answer still needs a careful prompt and source citation.

🧪 Ask one answerable question and one question not in the documents. The second one should refuse clearly.

`practice_05_local_doc_qa_cli.py`

Streamlit local document chatbot

The same logic is now wrapped in a UI.

```
with st.sidebar:
    st.header("Settings")
    chunk_size = st.slider("Chunk size", 400, 1600, 900, 100)
    overlap = st.slider("Overlap", 0, 400, 150, 50)
    top_k = st.slider("Retrieved chunks", 2, 8, 4)
    strict_mode = st.checkbox(
        "Strict mode: answer only from documents",
        value=True
    )

uploaded_files = st.file_uploader(
    "Upload one or more documents",
    type=["txt", "md", "py", "csv", "html", "pdf", "docx"],
    accept_multiple_files=True,
)

if uploaded_files and st.button("Build local index"):
    st.session_state.records = build_index(uploaded_files, chunk_size,
overlap)
```

Run:
streamlit run
practice_06_final_streamlit_local_doc_chatbot.py

practice_06_final_streamlit_local_doc_chatbot.py